

ASSIGNMENT NO. 11: Indexing and Query Optimization Queries

Questions

Q1) Create a MySQL database named SalesDB and a table named OrderData.

OrderData Table

Column Name	Data Type	Constraint
OrderID	INT	PRIMARY KEY
CustomerName	VARCHAR(50)	NOT NULL
Email	VARCHAR(80)	NOT NULL
City	VARCHAR(30)	NOT NULL
ProductCategory	VARCHAR(30)	NOT NULL
OrderDate	DATE	NOT NULL
OrderAmount	DECIMAL(10,2)	NOT NULL
OrderStatus	VARCHAR(20)	NOT NULL

Insert at least 12 records into the table.

Query-

```
CREATE DATABASE SalesDB;
```

```
USE SalesDB;
```

```
CREATE TABLE OrderData (  
    OrderID INT PRIMARY KEY,  
    CustomerName VARCHAR(50) NOT NULL,  
    Email VARCHAR(80) NOT NULL,  
    City VARCHAR(30) NOT NULL,  
    ProductCategory VARCHAR(30) NOT NULL,  
    OrderDate DATE NOT NULL,  
    OrderAmount DECIMAL(10,2) NOT NULL,  
    OrderStatus VARCHAR(20) NOT NULL  
);
```

```
INSERT INTO OrderData  
(OrderID, CustomerName, Email, City, ProductCategory, OrderDate, OrderAmount,  
OrderStatus)  
VALUES  
(1, 'Rahul Sharma', 'rahul@gmail.com', 'Pune', 'Electronics', '2026-01-10', 25000.00,  
'Delivered'),  
(2, 'Priya Patil', 'priya@gmail.com', 'Mumbai', 'Clothing', '2026-01-15', 4500.00,  
'Delivered'),  
(3, 'Amit Joshi', 'amit@gmail.com', 'Pune', 'Furniture', '2026-02-05', 18000.00, 'Pending'),  
(4, 'Neha Kulkarni', 'neha@gmail.com', 'Nashik', 'Electronics', '2026-02-12', 32000.00,  
'Shipped'),  
(5, 'Riya Deshmukh', 'riya@gmail.com', 'Mumbai', 'Books', '2026-02-20', 2200.00,  
'Delivered'),
```

(6, 'Karan Shah', 'karan@gmail.com', 'Pune', 'Clothing', '2026-03-03', 6500.00, 'Cancelled'),
 (7, 'Aarav Mehta', 'aarav@gmail.com', 'Nagpur', 'Electronics', '2026-03-10', 28000.00, 'Delivered'),
 (8, 'Sneha More', 'sneha@gmail.com', 'Pune', 'Books', '2026-03-18', 3500.00, 'Pending'),
 (9, 'Vikas Singh', 'vikas@gmail.com', 'Mumbai', 'Furniture', '2026-04-02', 22000.00, 'Shipped'),
 (10, 'Anjali Rao', 'anjali@gmail.com', 'Nashik', 'Clothing', '2026-04-10', 5500.00, 'Delivered'),
 (11, 'Rohan Gupta', 'rohan@gmail.com', 'Pune', 'Electronics', '2026-04-15', 41000.00, 'Pending'),
 (12, 'Pooja Nair', 'pooja@gmail.com', 'Nagpur', 'Books', '2026-04-22', 2800.00, 'Delivered');

Q2) Create the following MySQL indexes on the OrderData table:

Index Name	Column	Index Type
idx_city	City	Normal index
idx_email	Email	Unique index
idx_order_date	OrderDate	Normal index

Display all indexes using SHOW INDEX.

Query-

```
-- Select the database
USE SalesDB;
```

```
-- Create normal index on City
CREATE INDEX idx_city
ON OrderData(City);
```

```
-- Create unique index on Email
CREATE UNIQUE INDEX idx_email
ON OrderData(Email);
```

```
-- Create normal index on OrderDate
CREATE INDEX idx_order_date
ON OrderData(OrderDate);
```

```
-- Display all indexes
SHOW INDEX FROM OrderData;
```

Output-

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
orderdata	0	PRIMARY	1	OrderID	A	12	NULL	NULL		BTREE		
orderdata	0	idx_email	1	Email	A	12	NULL	NULL		BTREE		
orderdata	1	idx_city	1	City	A	12	NULL	NULL		BTREE		
orderdata	1	idx_order_date	1	OrderDate	A	12	NULL	NULL		BTREE		

Q3) Create a composite index named idx_city_order_date.

The index should contain:

Index Position	Column
1	City
2	OrderDate

Write a query that searches for Mumbai orders within a specified date range.

Query-

```
CREATE INDEX idx_city_order_date
ON OrderData (City, OrderDate);
```

```
SELECT OrderID, CustomerName, City, OrderDate, OrderAmount
FROM OrderData
WHERE City = 'Mumbai'
AND OrderDate BETWEEN '2026-01-01' AND '2026-03-31';
```

Output-

	OrderID	CustomerName	City	OrderDate	OrderAmount
☐ Edit Copy Delete	2	Priya Patil	Mumbai	2026-01-15	4500.00
☐ Edit Copy Delete	5	Riya Deshmukh	Mumbai	2026-02-20	2200.00

Q4) Write a MySQL EXPLAIN query to analyze the following search:

```
SELECT *
FROM OrderData
WHERE City = 'Mumbai';
```

Record and explain the following output columns:

EXPLAIN Column
type
possible_keys
key
rows
Extra

Query-

```
EXPLAIN
SELECT *
FROM OrderData
WHERE City = 'Mumbai';
```

Output-

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	OrderData	ref	idx_city,idx_city_order_date	idx_city	122	const	3	Using index condition

EXPLAIN Column Explanation

type	ref – MySQL uses a non-unique index to find the rows where City = 'Mumbai' instead of scanning the whole table.
possible_keys	idx_city, idx_city_order_date – both indexes start with the City column, so either can be used.
key	idx_city – the index actually chosen by the optimizer.
rows	3 – only about 3 rows are examined instead of all 12 rows.
Extra	Using index condition – the City filter is applied using the index before the full row is read from the table.

Q5) Compare query performance before and after creating an index on City.

Perform the following steps:

Step	Task
1	Drop idx_city, if it exists
2	Run EXPLAIN on the City search
3	Create idx_city
4	Run the same EXPLAIN query again
5	Compare the selected index and estimated rows

Query-

Step 1 Drop the index if it exists:

DROP INDEX idx_city ON OrderData;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

USE SalesDB;

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0222 seconds.)

DROP INDEX idx_city ON OrderData;

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

Step 2 - Run EXPLAIN without the index:

```
EXPLAIN
SELECT *
FROM OrderData
WHERE City = 'Mumbai';
```

Your SQL query has been executed successfully.

[EXPLAIN](#) [SELECT](#) * FROM OrderData WHERE City = 'Mumbai';

[\[Edit inline \]](#) [\[Edit \]](#) [\[Skip Explain SQL \]](#) [\[Create PHP code \]](#)

Extra options

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	OrderData	ref	idx_city_order_date	idx_city_order_date	122	const	3	Using index condition

Step 3- Create the index:

```
CREATE INDEX idx_city
ON OrderData (City);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0250 seconds.)

[CREATE](#) [INDEX](#) idx_city ON OrderData(City);

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Step 4 - Run the same EXPLAIN query again:

```
EXPLAIN
SELECT *
FROM OrderData
WHERE City = 'Mumbai';
```

Your SQL query has been executed successfully.

[EXPLAIN](#) [SELECT](#) * FROM OrderData WHERE City = 'Mumbai';

[\[Edit inline \]](#) [\[Edit \]](#) [\[Skip Explain SQL \]](#) [\[Create PHP code \]](#)

Extra options

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	OrderData	ref	idx_city_order_date,idx_city	idx_city_order_date	122	const	3	Using index condition

Step 5- Compare type, possible_keys, key, rows, and Extra. The exact plan can vary because MySQL's optimizer considers the table size, statistics, selectivity, and available indexes.

Column	Before Index	After Index
type	ref	ref
possible_keys	idx_city_order_date	idx_city_order_date, idx_city
key	idx_city_order_date	idx_city_order_date
rows	3	3
Extra	Using index condition	Using index condition

Conclusion: The composite index `idx_city_order_date` (created in Q3) has `City` as its first column, so MySQL can already use it for the search `City = 'Mumbai'`. Therefore, in both cases the optimizer selects `idx_city_order_date` with `type = ref` and about 3 estimated rows. Dropping `idx_city` before the test and re-creating it afterwards makes no visible difference, because a `ref` lookup on an index (3 rows) is already used instead of a full table scan (`ALL`, 12 rows).

Q6) Rewrite the following query in a more index-friendly form:

```
SELECT *
FROM OrderData
WHERE YEAR(OrderDate) = 2026;
```

The improved query should:

Requirement
Avoid applying a function to OrderDate
Use a start date
Use an end date
Select only OrderID, CustomerName, OrderDate, and OrderAmount
Be analyzed using EXPLAIN

Query-

```
SELECT OrderID, CustomerName, OrderDate, OrderAmount
FROM OrderData
WHERE OrderDate >= '2026-01-01'
AND OrderDate < '2027-01-01';
```

	OrderID	CustomerName	OrderDate	OrderAmount
☐ Edit Copy Delete	1	Rahul Sharma	2026-01-10	25000.00
☐ Edit Copy Delete	2	Priya Patil	2026-01-15	4500.00
☐ Edit Copy Delete	3	Amit Joshi	2026-02-05	18000.00
☐ Edit Copy Delete	4	Neha Kulkarni	2026-02-12	32000.00
☐ Edit Copy Delete	5	Riya Deshmukh	2026-02-20	2200.00
☐ Edit Copy Delete	6	Karan Shah	2026-03-03	6500.00
☐ Edit Copy Delete	7	Aarav Mehta	2026-03-10	28000.00
☐ Edit Copy Delete	8	Sneha More	2026-03-18	3500.00
☐ Edit Copy Delete	9	Vikas Singh	2026-04-02	22000.00
☐ Edit Copy Delete	10	Anjali Rao	2026-04-10	5500.00
☐ Edit Copy Delete	11	Rohan Gupta	2026-04-15	41000.00
☐ Edit Copy Delete	12	Pooja Nair	2026-04-22	2800.00

```
EXPLAIN
SELECT OrderID, CustomerName, OrderDate, OrderAmount
FROM OrderData
WHERE OrderDate >= '2026-01-01'
AND OrderDate < '2027-01-01';
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	OrderData	ALL	idx_order_date	NULL	NULL	NULL	12	Using where

.

Explanation: `YEAR(OrderDate) = 2026` applies a function to the column, so MySQL must calculate it for every row and cannot use the index on `OrderDate`. The rewritten query compares `OrderDate` directly with a start date (`>= '2026-01-01'`) and an end date (`< '2027-01-01'`), so a range scan on `idx_order_date` is possible. In the output above MySQL still shows `type = ALL` (12 rows), because every row in this small table falls in 2026 and a full scan is cheaper than using the index. On a large table with data from many years, this form of the query would use the index.